

NLP Search Engine

A small, hackable information–retrieval engine in pure Python

Hamza Abdul Karim · F23607046 — May 2026

ABSTRACT

This project implements the classical information-retrieval stack — tokenization, Porter stemming, an inverted index, and TF-IDF / Okapi BM25 ranking — in roughly two thousand lines of Python. The code is organised into single-responsibility files of fewer than two hundred lines each, so it can be read in one sitting. Features include semantic search with TF-IDF vectors, search analytics, query autocomplete, faceted search with filters, batch operations, and incremental indexing. Three front-ends share the same SearchEngine facade: a command-line interface, a Flask web application with dark/light theme toggle, and a comprehensive JSON API.

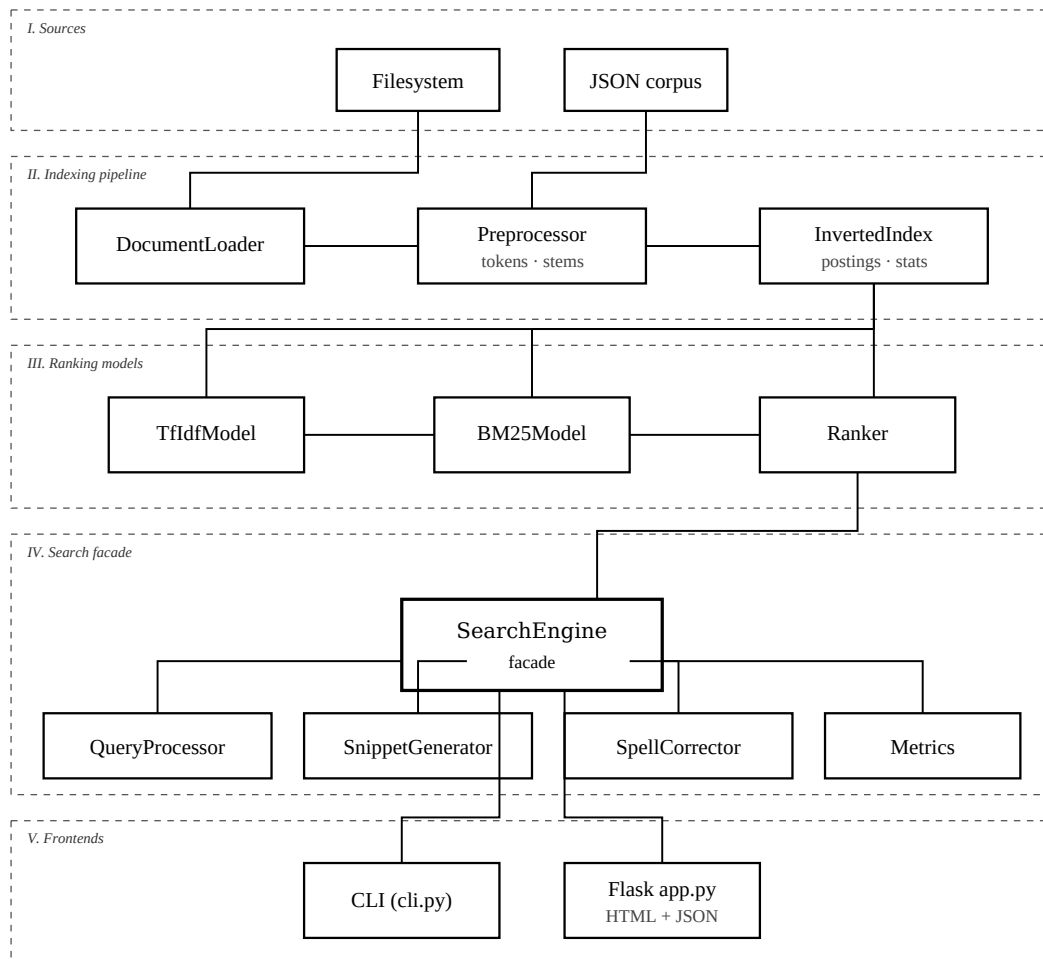
1. What's Inside

- NLP preprocessing — tokenization, case folding, stop-word removal, and a compact Porter stemmer.
- Inverted index — postings carry term frequencies and positions, enabling phrase queries.
- Multiple ranking models — TF-IDF, BM25, and semantic search with TF-IDF vectors.
- Smart query parsing — phrases, +must, –exclude, boolean operators (AND, OR, NOT).
- Snippets and highlights — best-window selection with matched-term highlighting.
- Spell suggestions — Damerau-Levenshtein matching against the index vocabulary.
- Search analytics — query tracking, top queries, related queries, and Prometheus metrics.
- Faceted search — filter by domain, source, score; sort by relevance, title, or score.
- Autocomplete — real-time suggestions as you type.
- Three front-ends — CLI, Flask web UI with theme toggle, bookmarks, history, and comprehensive JSON API.

2. Architecture at a Glance

The system is organised into five horizontal layers, shown in Figure 1. Inputs flow into the indexing pipeline; the resulting inverted index is consumed by two interchangeable ranking models, exposed through a thin Ranker facade. The SearchEngine ties everything together and is the single entry point for both the CLI and the web application. The full set of architectural diagrams is in [docs/architecture.pdf](#).

Figure 1. System architecture
Components and their compile-time dependencies



Solid arrows denote a "depends on" relation. Dashed boxes group components by responsibility.

Figure 1. System architecture (see docs/architecture.pdf for the full set).

3. Quickstart

```
git clone https://github.com/Hamza35779/NLP-AI-Search-Engine.git
cd NLP-AI-Search-Engine

pip install -r requirements.txt
python scripts/build_index.py

python cli.py "machine learning" # one-shot CLI
python cli.py # CLI in REPL mode
python app.py # Flask UI on :5000
```

Or with Docker:

```
docker build -t nlp-search-engine .
docker run --rm -p 5000:5000 nlp-search-engine
```

4. Demo Queries

```
information retrieval  
"natural language"  
ranking +bm25 -neural  
neural NOT toy
```

5. Project Layout

```
nlp-search-engine/  
app.py Flask web UI (with new endpoints)  
cli.py Argparse CLI  
requirements.txt Python dependencies  
pyproject.toml Packaging + tool config  
Dockerfile Production container  
Makefile Build automation  
README.md This file  
LICENSE MIT License  
■  
src/ Engine source (every file <200 lines)  
■ ■■■ search_engine.py Main facade (with caching, analytics, metrics)  
■ ■■■ document.py Document class (with URL/domain support)  
■ ■■■ document_loader.py Corpus loading + web crawling  
■ ■■■ indexer.py Inverted index  
■ ■■■ ranker.py Ranking model wrapper  
■ ■■■ bm25.py BM25 implementation  
■ ■■■ tfidf.py TF-IDF implementation  
■ ■■■ semantic_search.py Semantic search (TF-IDF vectors)  
■ ■■■ preprocessing.py Tokenization, stemming, stopwords  
■ ■■■ query_processor.py Query parsing  
■ ■■■ snippet.py Snippet generation  
■ ■■■ spell_check.py Spell checker  
■ ■■■ cache.py Search result caching layer  
■ ■■■ logger.py Logging configuration  
■ ■■■ metrics.py IR metrics + SearchMetrics  
■ ■■■ analytics.py Search analytics tracking  
■ ■■■ autocomplete.py Query autocomplete suggestions  
■ ■■■ persistence.py SQLite backend (optional)  
■ ■■■ config.py Configuration (v0.3.0)  
■ ■■■ utils.py Helper functions  
■  
templates/ Jinja2 HTML templates  
■ ■■■ base.html Base layout (history, bookmarks, theme)  
■ ■■■ index.html Home page with hero section  
■ ■■■ results.html Search results (facets, bookmarks)  
■ ■■■ about.html Project information  
■ ■■■ _partials/ Reusable components  
■  
static/ CSS, JS, and images  
■ ■■■ css/  
■ ■ ■■■ base.css Design tokens, typography, layout (v2)  
■ ■ ■■■ components.css Search form, cards, badges (v2)  
■ ■ ■■■ home.css Hero, features, steps  
■ ■ ■■■ results.css Results page layout (v2)  
■ ■■■ js/  
■ ■ ■■■ app.js Search history, bookmarks, theme toggle  
■ ■■■ img/ Logos and favicons  
■  
tests/ pytest test suite (25 tests)  
scripts/ Build/indexing scripts  
examples/ Usage examples  
■■■ data/ Sample corpus  
■■■ sample_docs/ 7 NLP/IR documents
```

6. Documentation

Topic	Location
System overview and diagrams	docs/architecture.pdf
TF-IDF and BM25 in depth	docs/ranking-models.pdf
Semantic search with TF-IDF vectors	docs/semantic-search.pdf
HTTP/JSON API (full reference)	docs/api.pdf
Search analytics and metrics	docs/analytics.pdf
Dev environment and contributing	docs/development.pdf and CONTRIBUTING.pdf
Release history	CHANGELOG.pdf

7. Testing

```
pytest -q
```

The suite is fast (well under a second) and covers preprocessing, the inverted index, both rankers, the query parser, snippets, spell suggestions, and the IR metric helpers.

8. New Features in v0.3.0

- Semantic Search — TF-IDF vector embeddings for semantic matching.
- Search Analytics — query tracking, top queries, related queries.
- Query Autocomplete — real-time suggestions as you type.
- Faceted Search — filter by domain, source, score; sort options.
- Search History & Bookmarks — LocalStorage with theme toggle.
- Batch Search API — process up to 20 queries at once.
- Metrics & Monitoring — Prometheus endpoint, health checks.
- Incremental Indexing — SQLite backend for efficient updates.

9. License

MIT © 2026 Hamza Abdul Karim. See the bundled LICENSE file for full text.